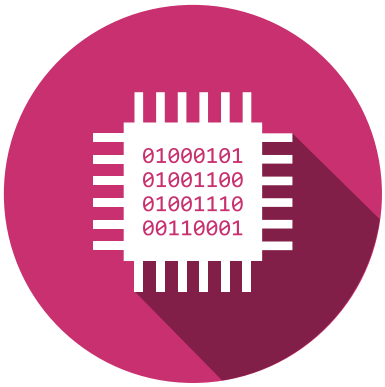
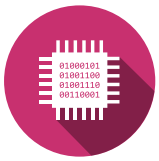


Machines d'état (FSM)

Course Conception Numérique (CNum)

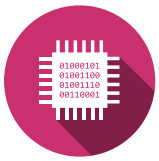
Hes·so VALAIS WALLIS
Haute Ecole d'Ingénierie
Hochschule für Ingenieurwissenschaften

Orientation: Systèmes industriels (Synd)
Spécialisation: Infotronics (IT)
Cours: Conception Numérique (CNum)
Auteurs: Silvan Zahno, Christophe Bianchi, François Corthay, Pierre Pompili
Date: 03.11.2026



Contenu

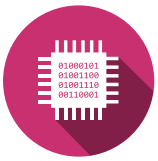
1 Introduction 3
2 Systèmes logiques synchrones 4
2.1 Définition: système logique synchrone 4
2.2 Signal d'horloge 4
3 Machines de Moore 5
3.1 Architecture 5
3.1.1 Définition: input state (état d'entrée) 5
3.1.2 Définition: internal state (état interne) 5
3.1.3 Définition: état de sortie 5
3.2 Table de vérité des blocs combinatoires 5
Exemple: compteur cascadable 6
3.3 Graphe des états 6
Exemple: compteur cascadable 6
3.4 Définition: état total stable 7
4 Machines de Mealy 8
4.1 Architecture 8
4.2 Comportement temporel 8
4.3 Graphe des états 8
Exemple: compteur cascadable avec inhibition du report 8
5 Établissement du graphe des états 10
5.1 Conception de Finite State Machine, FSM (machine d'état) 10
5.2 Développement à partir d'un état quelconque 10
Exemple: circuit anti-rebonds 10
5.3 Développement à partir d'un scénario 12
Exemple: circuit anti-rebonds 12
5.4 Développement à partir de la liste des états 13
5.4.1 Exemple: circuit anti-rebonds 13
6 Réduction de graphes 15
6.1 Graphes d'états équivalents 15
6.2 Table d'états 15
Exemple: circuit anti-rebonds 15
6.3 Méthode de réduction 15
Exemple: circuit anti-rebonds 16
7 Codage des états 17
7.1 Codage minimal 17
Exemple: circuit anti-rebonds 17
7.2 Codage 1 parmi m 17
Exemple: circuit anti-rebonds 18
Glossaire 19



1 Introduction

Les circuits de commande se font généralement à l'aide de **machines d'états**.

On distingue les machines de Moore, où les sorties dépendent uniquement des états internes, et les machines de Mealy, où les sorties dépendent des états internes et des entrées.



2 Systèmes logiques synchrones

2.1 Définition: système logique synchrone

Un système logique synchrone est caractérisé par le fait que toutes les sorties des bascules peuvent uniquement changer au moment d'un flanc de l'horloge maîtresse du système. Ceci implique que

- toutes les bascules ont le même signal d'horloge,
- aucune bascule n'a d'entrées asynchrones.

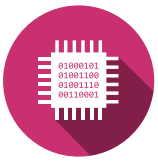


Pour des raisons d'initialisation et de testabilité, il est usuel de connecter une commande de mise à zéro à toutes les bascules. Dans le cas où la commande de remise à zéro provient de l'extérieur du circuit et est identique pour toutes les bascules, on considère tout de même le système comme synchrone.

2.2 Signal d'horloge

Le signal d'horloge provient d'un oscillateur. Il a une fréquence régulière et n'est pas à bloquer par des portes logiques dans le but d'arrêter le circuit pendant un certain temps.

Généralement, le signal d'horloge a une fréquence bien plus élevée que celle des signaux d'entrée du système logique synchrone.



3 Machines de Moore

3.1 Architecture

Une Finite State Machine, FSM (machine d'état) de Moore est un système logique synchrone comprenant

- des bascules qui mémorisent un état interne,
- un bloc logique combinatoire qui détermine l'état interne futur en fonction de l'état interne présent et des entrées,
- un bloc logique combinatoire qui détermine la valeur des sorties en fonction de l'état interne présent.

La Figure 1 représente un exemple de machine d'état de Moore.

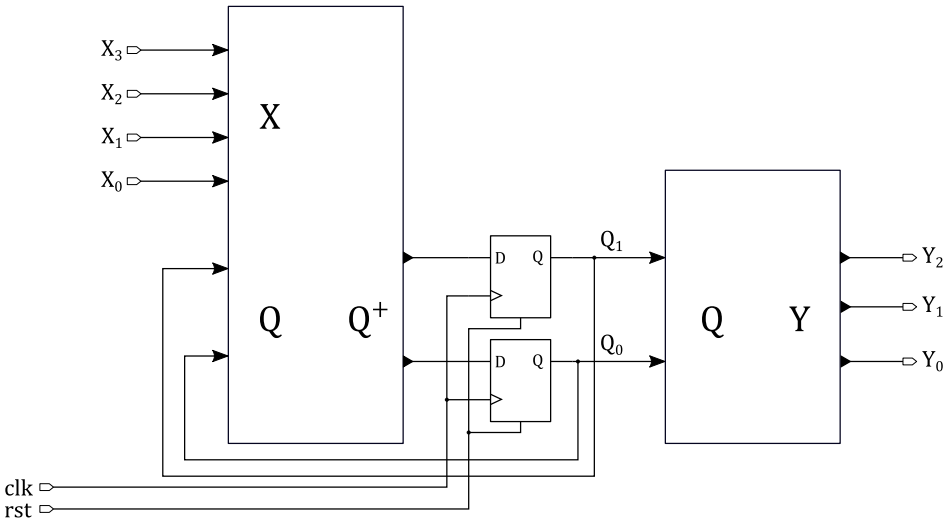


Figure 1 - Architecture d'une machine d'état de Moore

3.1.1 Définition: état d'entrée

Un input state (état d'entrée) est une combinaison de valeurs des signaux d'entrée.

Pour un système à n entrées, il y a 2^n états d'entrée possibles.

3.1.2 Définition: état interne

Un internal state (état interne) est une combinaison de valeurs des sorties des bascules.

Pour un système à m bascules, il y a 2^m états internes possibles.

3.1.3 Définition: état de sortie

Un output state (état de sortie) est une combinaison de valeurs des signaux de sortie.

Pour un système à l sorties, il y a 2^l états de sortie possibles.

3.2 Table de vérité des blocs combinatoires

Le fonctionnement d'une machine d'état peut s'étudier à l'aide d'une table de vérité qui donne, pour chaque état d'entrée et chaque état interne présent, l'état interne futur et l'état de sortie.



Exemple: compteur cascadable

La Figure 2 présente le schéma d'un compteur cascadable.

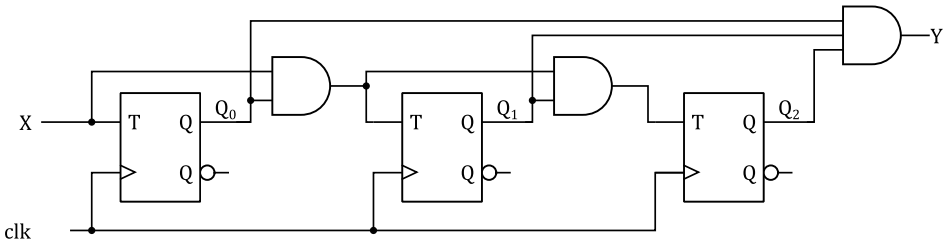


Figure 2 - Compteur cascadable

La Table 1 est la table de vérité correspondante.

X	$Q_{2...0}$	$T_{2...0}$	$Q^+_{2...0}$	Y
0	000	000	000	0
0	001	000	001	0
0	010	000	010	0
0	011	000	011	0
0	100	000	100	0
0	101	000	101	0
0	110	000	110	0
0	111	000	111	1
1	000	001	001	0
1	001	011	010	0
1	010	001	011	0
1	011	111	100	0
1	100	001	101	0
1	101	011	110	0
1	110	001	111	0
1	111	111	000	1

Table 1 - Table de vérité des blocs combinatoires du compteur cascadable

3.3 Graphe des états

La table de vérité des blocs combinatoires permet de dessiner le graphe des états. Celui-ci donne une représentation plus compréhensible du fonctionnement de la machine d'états.

Une machine d'états à m bascules comprend 2^m états. Chaque état est représenté par un cercle. Si la machine possède n entrées, il y a 2^n cas à spécifier: de chaque état partent 2^n flèches. Pour une machine de Moore, les sorties dépendent uniquement des états. Le plus simple est donc de représenter la valeur que prennent les sorties dans le cercle correspondant à l'état.

Exemple: compteur cascadable

Le graphe des états du compteur cascadable, donné à la Figure 3, se développe à partir de la table de vérité des blocs combinatoires donnée à la Table 1.

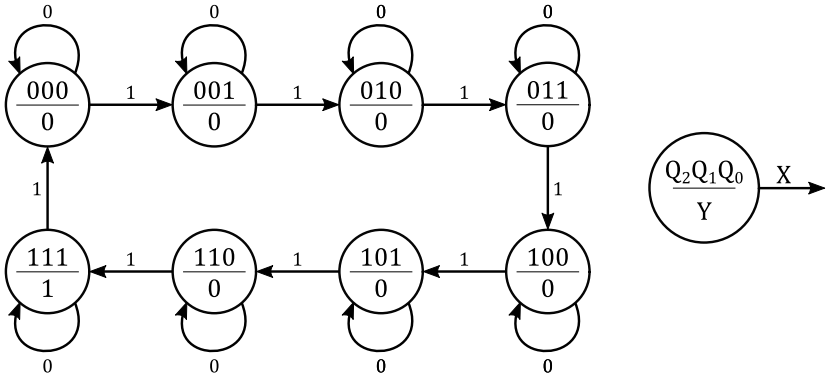


Figure 3 - Graphe des états du compteur cascadable

On observe qu'avec $X =$ '0', le compteur reste arrêté et qu'avec $X =$ '1' il tourne régulièrement. Lorsqu'il arrive à la valeur maximale, le compteur fait un report $Y =$ '1' pour la mise en cascade.



Pour la représentation du graphe des états, il est important de spécifier l'ordre dans lequel les signaux sont annotés.

3.4 Définition: état total stable

Un état total stable est un état dans lequel la machine reste arrêtée, pour un des états d'entrée au moins. Dans le graphe des états, d'un état total stable part une flèche qui revient sur lui-même.

4 Machines de Mealy

4.1 Architecture

A la différence des machines de Moore, les machines de Mealy ont des sorties qui peuvent dépendre aussi des entrées. La Figure 4 représente un exemple de machine d'état de Mealy.

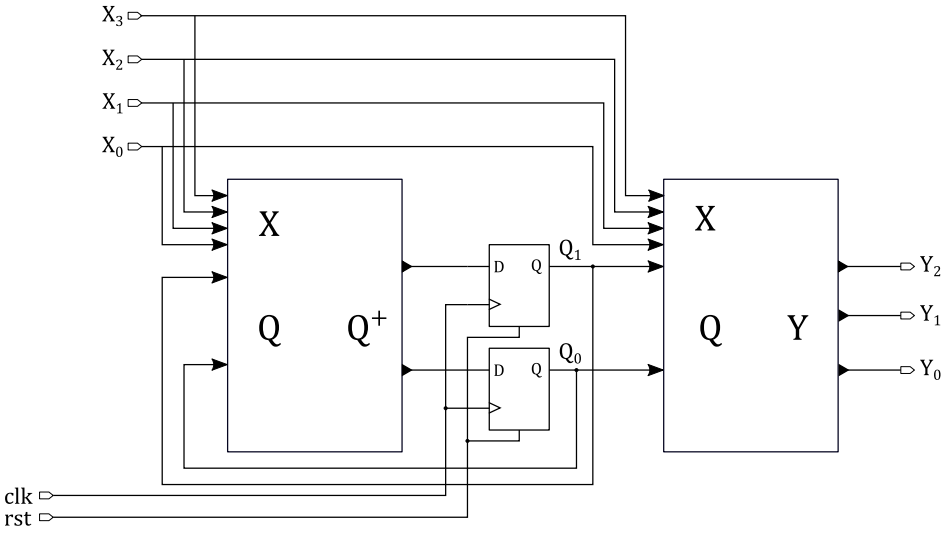


Figure 4 - Architecture d'une machine d'état de Mealy

4.2 Comportement temporel

La Figure 5 présente le comportement temporel des signaux de sortie d'une machine de Mealy.

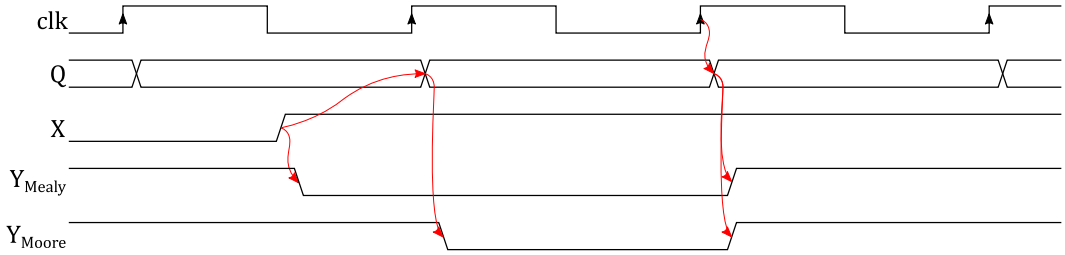


Figure 5 - Comportement temporel d'une machine de Mealy

Avec une machine de Mealy, les sorties peuvent réagir directement à la variation d'une entrée, alors qu'avec une machine de Moore elles ne changent qu'après un flanc actif de l'horloge.

4.3 Graphe des états

Le graphe des états d'une machine de Mealy doit montrer comment les sorties dépendent aussi des entrées. On y représente la valeur des sorties à côté de celle des entrées.

Exemple: compteur cascadable avec inhibition du report

La Figure 6 présente un compteur cascadable où le signal de report est mis à $Y = $ '0' lorsque l'entrée est à $X = $ '0'.

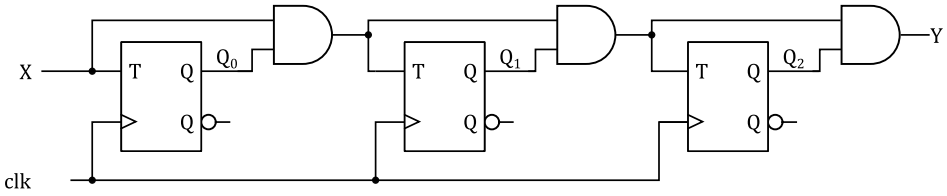


Figure 6 - Compteur cascadable avec inhibition du report

Le graphe des états correspondant est donné à la Figure 7.

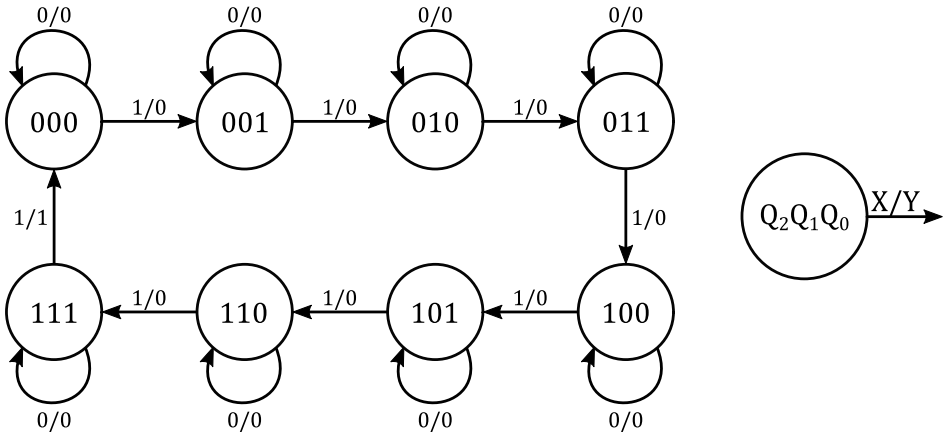
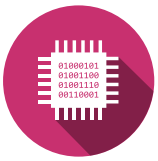


Figure 7 - Graphe des états du compteur cascadable avec inhibition du report



La valeur des sorties indiquée à côté d'une flèche est valable dans l'état d'où part la flèche et non dans celui où elle va.



5 Établissement du graphe des états

5.1 Conception de machine d'état

La conception de machine d'état peut se faire comme suit:

- expression du cahier des charges par un graphe d'états,
- remplissage de la table des états,
- réduction du nombre d'états,
- codage des états,
- réalisation du circuit logique.

Le point le plus délicat consiste à développer un graphe d'états à partir du cahier des charges du système. A cet effet, 3 méthodes sont présentées ci-après.

5.2 Développement à partir d'un état quelconque

La première manière de développer un graphe d'états se fait en procédant état par état. Pour cela, il faut

- Spécifier un état déterminé de la machine.
- Pour cet état, analyser toutes les conditions d'entrées possibles (2^n cas pour n entrées) et dessiner les flèches correspondantes. Selon les cas, de nouveaux états apparaissent.
- Prendre un par un les états non encore traités, analyser toutes les conditions d'entrées possibles (2^n cas pour n entrées) et dessiner les flèches correspondantes. Chaque fois qu'un nouvel état apparaît, vérifier qu'il n'existe pas déjà dans le graphe en cours de construction. Répéter cette étape jusqu'à ce que le graphe soit complet.



Il peut exister des cas où certaines conditions d'entrée ne peuvent se produire. Ceci est typiquement le cas si, par construction, plusieurs signaux d'entrée ne peuvent changer simultanément. Dans ce cas, on a moins 2^n de flèches qui partent de chaque état.

Exemple: circuit anti-rebonds

Le circuit anti-rebonds à concevoir est donné par le cahier des charges suivant: le circuit ne transmet le signal d'entrée que lorsque celui-ci n'a pas varié pendant les derniers trois coups d'horloge, sinon le circuit conserve la même valeur de sortie que précédemment.

Clairement, ceci est une machine de Moore: une variation a court terme de l'entrée ne doit aucunement affecter la sortie.

Spécifions un état déterminé de la machine: le signal d'entrée était à `'0'` pendant suffisamment longtemps, et la sortie est à `'0'`.



Figure 8 - Développement du graphe: état initial

L'analyse de toutes les conditions d'entrée possibles donne 2 cas: soit le signal d'entrée est à **`'0'`**, et on reste dans le même état, soit le signal est à **`'1'`**, et on change d'état pour se rapprocher de l'état où le signal de sortie pourra basculer à **`'1'`**.

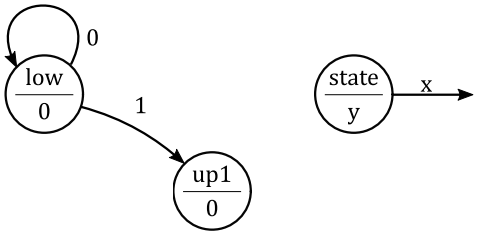


Figure 9 - Développement du graphe: analyse de toutes les conditions d'entrée

Un nouvel état est apparu. Il faut donc à nouveau analyser toutes les conditions d'entrée possibles pour ce nouvel état.

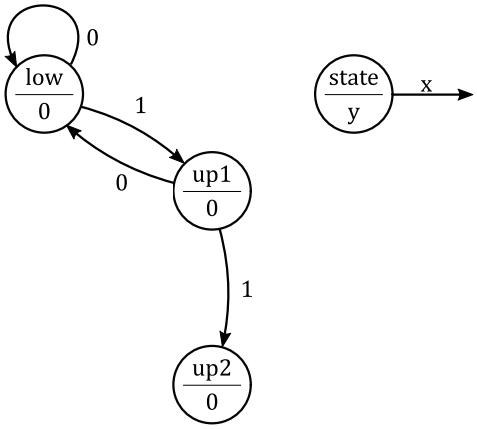


Figure 10 - Développement du graphe, suite

Par le développement successif des différents états avec l'analyse de toutes les conditions d'entrée possibles, on obtient le graphe de la Figure 11

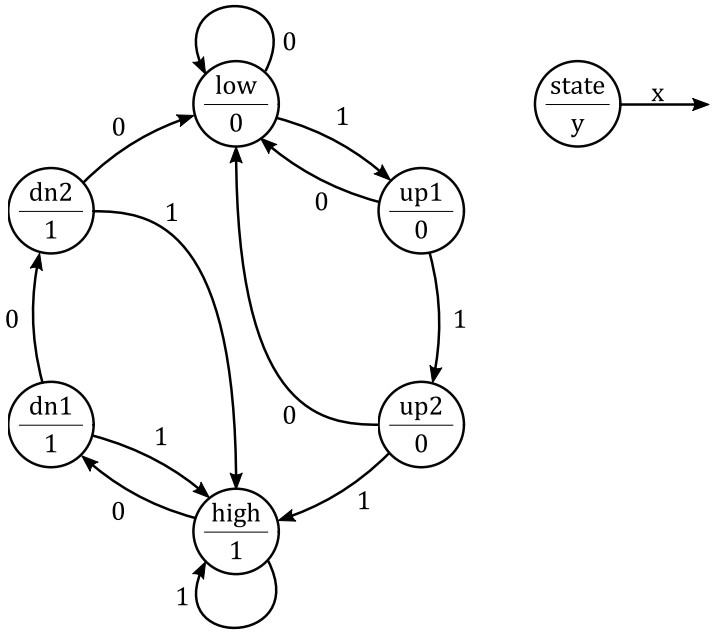


Figure 11 - Graphe des états du circuit anti-rebonds

5.3 Développement à partir d'un scénario

La deuxième manière de développer un graphe d'états se base sur la définition d'un scénario. Pour cela, il faut

- Définir un scénario couvrant une large part du cahier des charges.
- Développer le scénario sous forme de graphe, avec les états correspondant aux étapes du scénario et les flèches correspondant aux actions.
- Pour chaque état ainsi défini, analyser toutes les conditions d'entrées possibles (2^n cas pour n entrées) et dessiner les flèches non encore présentes. Parfois, de nouveaux états apparaissent. Il faut alors aussi faire l'analyse complète de ces nouveaux états.

Exemple: circuit anti-rebonds

Un scénario simple pour le développement du graphe du circuit anti-rebonds est le suivant: le signal d'entrée reste à `'0'` pendant suffisamment longtemps pour que la sortie soit à `'0'`, le signal passe à `'1'`, y reste suffisamment longtemps pour que la sortie le suive, puis il retombe à `'0'`. Ceci s'exprime par le graphe de la Figure 12.

Le graphe n'est pas encore complet. Il reste à analyser pour chacun des états toutes les conditions d'entrées possibles et à compléter le graphe en conséquence. On obtient alors le graphe de la Figure 11.

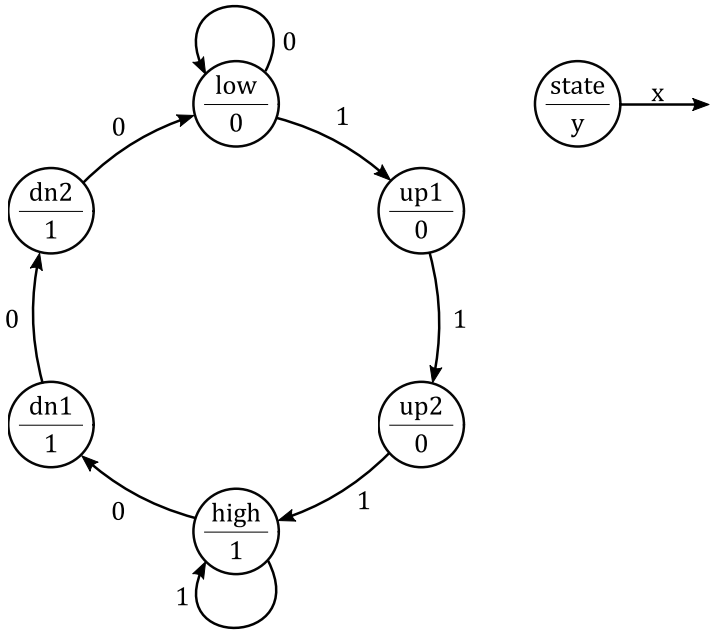


Figure 12 - Scénario de fonctionnement du circuit anti-rebonds

5.4 Développement à partir de la liste des états

La troisième manière de développer un graphe d'états se base sur la connaissance préalable de ses états. Pour cela, il faut

- Déterminer les événements ou les états à mémoriser.
- Pour chaque événement à mémoriser, dessiner un état.
- Pour chaque état ainsi défini, analyser toutes les conditions d'entrées possibles (2^n cas pour n entrées) et dessiner les flèches correspondantes.

5.4.1 Exemple: circuit anti-rebonds

Déterminons les choses à mémoriser: pour savoir si le signal n'a pas varié, il faut connaître les 2 dernières valeurs du signal d'entrée en plus de sa valeur présente; aussi, si le signal a varié, il faut se rappeler de la valeur précedente de la sortie. Il faut donc se souvenir de 3 valeurs binaires, ce qui nécessite 8 états, représentés à la Figure 13.

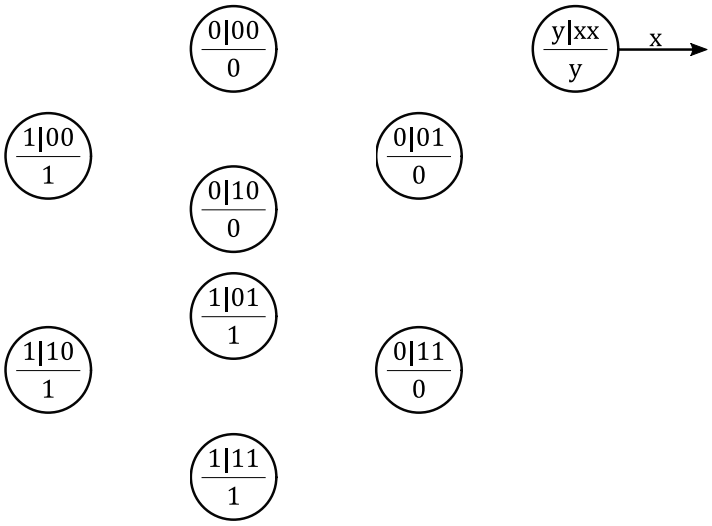


Figure 13 - Evénements à mémoriser

Pour chaque état ainsi défini, il faut analyser toutes les conditions d'entrées possibles et dessiner les flèches correspondantes. Ceci donne le graphe de la Figure 14.

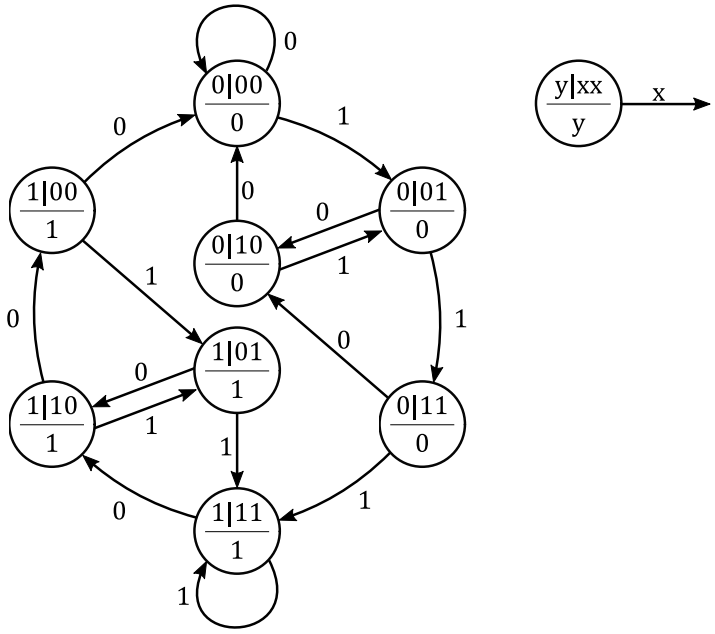
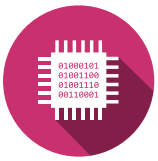


Figure 14 - Graphe des états à partir des événements à mémoriser



Les graphes développés à partir de différentes méthodes n'ont pas nécessairement le même nombre d'états, même si leur fonctionnement est identique.



6 Réduction de graphes

6.1 Graphes d'états équivalents

Deux graphes peuvent avoir un nombre d'états différent et présenter un fonction- nement identique.

Ceci est généralement dû au fait qu'un état peut être développé en 2 ou plusieurs états distincts. Inversément, il arrive que 2 ou plusieurs états puissent être simplifiés en un seul.

La méthode de réduction présentée ci-après de graphes sert à détecter ces répétitions et à simplifier ainsi les graphes d'état. Elle se base sur la description d'une machine d'états par une table d'états.

6.2 Table d'états

La table d'états donne, en fonction des entrées et des états présents, les états futurs et les sorties présentes.

Dans la table d'états, les états d'entrée spécifient les colonnes et les états internes spécifient les lignes.

On tâchera de placer les états d'entrée dans le même ordre que cela se fait dans une table de Karnaugh. Ceci servira à remplir plus aisément les tables de Karnaugh des blocs logiques combinatoires.



Pour les cas où certaines conditions d'entrée ne peuvent se produire, la table d'états est à remplir avec des états `'-'`.

Exemple: circuit anti-rebonds

La table d'états du circuit anti-rebonds de la Figure 14 est donnée à la Table 2.

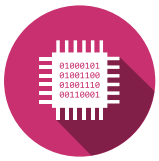
$Q \setminus x$	0	1
0 | 00	0 | 00/0	0 | 01/0
0 | 01	0 | 10/0	0 | 11/0
0 | 10	0 | 00/0	0 | 01/0
0 | 11	0 | 10/0	1 | 11/0
1 | 00	1 | 00/1	1 | 01/1
1 | 01	1 | 10/1	1 | 11/1
1 | 10	1 | 00/1	1 | 01/1
1 | 11	1 | 10/1	1 | 11/1

Table 2 - Table d'états du circuit anti-rebonds

6.3 Méthode de réduction

La technique de réduction consiste à relever dans la table d'états les lignes équivalentes et à les réduire en une seule.

Deux lignes sont équivalentes si



- elles ont dans chaque colonne des états futurs et des sorties présentes identiques,
- elles deviennent identiques si on remplace l'état qui spécifie une ligne par celui qui spécifie l'autre.

Exemple: circuit anti-rebonds

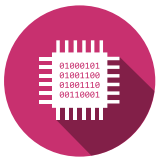
Dans la Table 2, la 1ère et la 3ème ligne sont équivalentes. De même, la 6ème et la 8ème ligne sont équivalentes. On peut donc considérer que l'état $0 \mid 00$ et l'état $0 \mid 10$ sont équivalents. Nous pouvons donc les renommer comme low. De même, l'état $1 \mid 01$ et l'état $1 \mid 11$ sont équivalents. Nous pouvons donc les renommer comme $high$.

En renommant les états $0 \mid 01$ en $up1$, $0 \mid 11$ en $up2$, $1 \mid 10$ en $dn1$ et $1 \mid 00$ en $dn2$, on obtient la Table 3.

$Q \setminus x$	0	1
low	$low/0$	$up1/0$
$up1$	$low/0$	$up2/0$
$up2$	$low/0$	$high/0$
$high$	$dn1/1$	$high/1$
$dn1$	$dn2/1$	$high/1$
$dn2$	$low/1$	$high/1$

Table 3 - Table d'états réduite du circuit anti-rebonds

On a ainsi obtenu la table d'états du graphe de la Figure 11. On voit donc bien que les 2 graphes sont équivalents.



7 Codage des états

Après réduction du graphe des états, il faut encore attribuer un code binaire aux différents états pour pouvoir remplir la table de vérité des blocs combinatoires et déterminer le schéma de la machine d'états.

7.1 Codage minimal

Le nombre minimal de bits, m, nécessaire pour coder les M états d'une machine est donné par

$$2^{m-1} < M < 2^m \tag{1}$$

Cette inéquation donne le nombre minimal de bascules nécessaire à la réalisation de la machine d'états.

L'assignement des codes peut se faire de manière aléatoire, en assignant le premier code binaire au premier état venu, le deuxième au suivant, etc.

Exemple: circuit anti-rebonds

La Table 4 donne un exemple de codage des états pour le circuits anti-rebonds.

$state$	Q
low	000
$up1$	001
$up2$	010
$high$	111
$dn1$	110
$dn2$	011

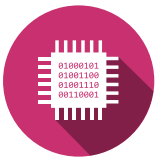
Table 4 - Codage binaire des états

Avec ce codage, on obtient les équations suivantes pour les blocs logiques combinatoires de la machine d'états:

$$\begin{cases} D_2 = xQ_1 + Q_2Q_0 \\ D_1 = xQ_1 + x\overline{Q_0} + Q_2 \\ D_0 = xQ_1 + x\overline{Q_0} + Q_2\overline{Q_0} \\ Y \ \ = Q_2 + Q_1Q_0 \end{cases} \tag{2}$$

7.2 Codage 1 parmi m

Une autre technique de codage consiste à utiliser M bits, et donc M bascules, pour coder les M états d'une machine. Le codage est fait de sorte que pour chaque état un seul des bits soit à `'1'`.





Ce codage est gourmand en bascules, mais les fonctions combinatoires qui y sont associées sont généralement très simples.

Surtout, il permet un décodage des états sans aléa. Ce point est d'une importance particulières pour des machines d'états qui régissent un séquencement d'opérations.

Dans un code 1 parmi m, la majorité des états internes possibles ne sont pas utilisés. La probabilité de voir apparaître des boucles indésirables en plus de la boucle principale du graphe est donc élevée. Il est donc important de prévoir une logique pour mettre la machine dans un des états de la boucle principale.

Exemple: circuit anti-rebonds

La Table 5 donne un exemple de codage 1 parmi m pour les états du graphe de la Figure 11.

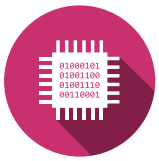
$state$	$Q_5 ... Q_0$
low	000001
$up1$	000010
$up2$	000100
$high$	001000
$dn1$	010000
$dn2$	100000

Table 5 - Codage 1 parmi m

On peut alors directement déduire les équations du graphe des états. En effet, chaque état est associé à un bit d'état. La condition «l'état futur est $up1$ si l'état présent est low et l'entrée vaut `'1'`» s'écrit $D_1 = Q_0 x$. Les équations pour les blocs logiques combinatoires de la machine d'états sont donc donnés par

$$\begin{cases} D_0 = \bar{x}Q_0 + \bar{x}Q_1 + \bar{x}Q_2 + \bar{x}Q_5 \\ D_1 = xQ_0 \\ D_2 = xQ_1 \\ D_3 = xQ_2 + xQ_3 + xQ_4 + xQ_5 \\ D_4 = \bar{x}Q_3 \\ D_5 = \bar{x}Q_4 \\ Y \ = Q_3 + Q_4 + Q_5 \end{cases} \tag{3}$$

Pour l'initialisation, on doit prévoir de mettre toutes les bascules à `'0'` sauf une qui est à mettre à `'1'`.



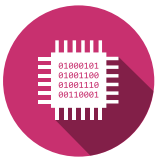
Glossaire

état de sortie – **output state** 5, 5

état d'entrée – **input state** 2, 5, 5, 5

état interne – **internal state** 2, 5, 5, 5, 5

machine d'état – **Finite State Machine, FSM** 2, 5, 5, 5, 5, 6, 6, 8, 10, 10



Bibliographie

[1] D. Mange, *Analyse et synthèse des systèmes logiques*, Editions Géorgi., vol. Traité d'électricité, volume V. St Saphorin: PPUR presses polytechniques, 1995.

[2] R. H. Katz et G. Borriello, *Contemporary Logic Design*. California: The Benjamin/Cummings Publishing Company Inc, 2005.

[3] D. Lewin et D. Protheroe, *Design of Logic Systems*, Second edition. Hong Kong: Springer, 2013.

[4] M. D. Ciletti et M. M. Mano, *Digital Design*, Second edition. New-Jersey: Prentice-Hall, 2007.

[5] J. F. Wakerly, *Digital Design: Principles And Practices*, 3rd edition. Prentice-Hall, 2008.

[6] D. J. Comer, *Digital Logic and State Machine Design*. Saunders College Publishing, 1995.

[7] S. Almani, *Electronic Logic Systems*, Second edition. New-Jersey: Prentice-Hall, 1989.

[8] M. V. Künzli et M. Meli, *Vom Gatter Zu VHDL: Eine Einführung in Die Digitaltechnik*. vdf Hochschulverlag AG, 2007.